# Lecture Notes: Logistic Regression

CIC1205 - Machine Learning

Eduardo Bezerra

March 4, 2026

# 0 Contents

**1 Introduction** **2**
1.1 From Regression to Classification . . . 2
1.2 Why Not Use Linear Regression for Classification? . . . 2
1.3 The Learning Pipeline (Revisited) . . . 2

**2 Representation of the Hypothesis** **3**
2.1 Notation . . . 3
2.2 The Sigmoid Function . . . 3
2.3 Odds, Log-Odds, and Logits . . . 4
2.4 The Hypothesis Function . . . 6
2.5 Decision Boundary . . . 6

**3 A Worked Example: Tumour Classification** **7**
3.1 Dataset and Learned Parameters . . . 7
3.2 Step 1: Locate the Decision Boundary . . . 7
3.3 Step 2: Compute the Linear Combination $z$ . . . 8
3.4 Step 3: Apply the Sigmoid Function . . . 8
3.5 Step 4: Apply the Decision Threshold . . . 8

**4 The Cost Function** **9**
4.1 Why Not Use the MSE? . . . 9
4.2 The Log-Loss (Binary Cross-Entropy) . . . 9
4.3 The Full Cost Function . . . 10
4.4 Optimisation Objective . . . 10

**5 Parameter Learning: Gradient Descent** **11**
5.1 Computing the Gradient . . . 11
5.2 The Gradient Descent Algorithm . . . 11
5.3 Worked Example: Single Update Step . . . 12

**6 The Decision Boundary in Detail** **13**
6.1 Linear Decision Boundary . . . 13
6.2 Non-Linear Decision Boundaries . . . 13

**7 Regularisation** **14**
7.1 The Problem of Overfitting . . . 14
7.2 Regularisation . . . 14
7.3 Regularised Gradient Descent . . . 15

**8 Prediction and Evaluation** **15**
8.1 Making Predictions . . . 15
8.2 Evaluation Metrics . . . 15

**9 Summary** **16**

**A Appendix: Deriving the Gradient of the Log-Loss** **17**

# 1 Introduction

## 1.1 From Regression to Classification

In the previous set of notes, we studied **linear regression**, whose goal is to predict a *continuous* numerical output from one or more input features. Now we turn to a different, but equally fundamental, problem: **classification**.

In a **binary classification** problem, the target variable $y$ can take only two values, which we conventionally encode as $y \in \{0, 1\}$. The value 1 typically represents the presence of some condition (the *positive class*), and 0 its absence (the *negative class*). Classic examples include:

- Predicting whether a tumour is malignant ($y = 1$) or benign ($y = 0$).
- Deciding whether an email is spam ($y = 1$) or not spam ($y = 0$).
- Determining whether a credit application should be approved ($y = 1$) or rejected ($y = 0$).

## 1.2 Why Not Use Linear Regression for Classification?

A natural first attempt might be to apply linear regression directly: fit $h_\theta(x) = \theta_0 + \theta_1 x$ and threshold the output (e.g., predict $\hat{y} = 1$ if $h_\theta(x) \geq 0.5$). This approach has two serious problems.

First, the predictions are not bounded: linear regression can output values well above 1 or well below 0, which are meaningless as probabilities. Second, the presence of a single outlier far from the decision boundary can dramatically shift the regression line, distorting the classification threshold.

What we really want is a model whose output is naturally *constrained to the interval* $[0, 1]$ so that it can be interpreted as a probability. This is precisely what **logistic regression** provides.

**Logistic Regression**

**Logistic Regression** is a supervised learning algorithm for binary classification. It models the probability that an input $x$ belongs to the positive class:

$$\Pr(y = 1 \mid x; \theta)$$

by applying a non-linear *sigmoid* function to a linear combination of the input features, ensuring that all outputs lie in $(0, 1)$.

## 1.3 The Learning Pipeline (Revisited)

The general supervised learning pipeline still applies, but with adjustments appropriate to classification:

1. **Collect a training set**: labelled examples $(x^{(i)}, y^{(i)})$ where $y^{(i)} \in \{0, 1\}$.
2. **Choose a hypothesis class**: a function that outputs a probability in $(0, 1)$.

3. **Define a cost function**: measure how well the predicted probabilities match the true labels.
4. **Optimise the parameters**: minimise the cost using gradient descent.
5. **Predict**: threshold the probability to produce a binary class label.

# 2 Representation of the Hypothesis

## 2.1 Notation

We extend the notation used in the linear regression notes.

| Symbol | Meaning |
|---|---|
| $m$ | Number of training examples |
| $n$ | Number of features |
| $x^{(i)}$ | Feature vector of the $i$-th training example |
| $y^{(i)}$ | Label of the $i$-th training example (0 or 1) |
| $(x^{(i)}, y^{(i)})$ | The $i$-th training example |
| $\theta_0, \theta_1, \ldots, \theta_n$ | Model parameters |
| $z$ | Linear combination $\theta^T x = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$ |
| $\sigma(z)$ | Sigmoid function applied to $z$ |
| $h_\theta(x)$ | Hypothesis function: estimated probability $\Pr(y = 1 \mid x; \theta)$ |

## 2.2 The Sigmoid Function

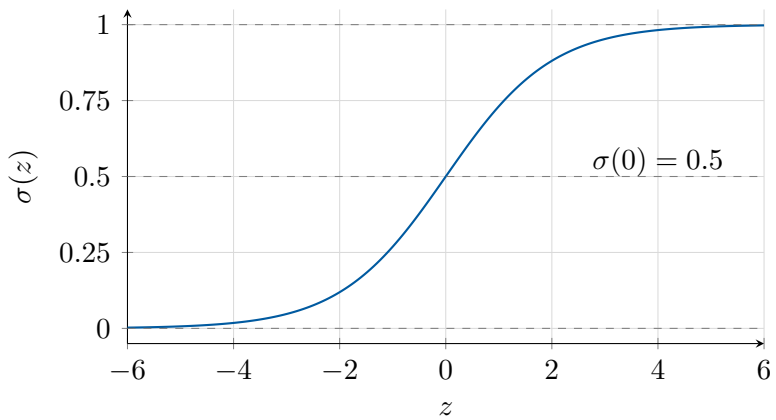
The key ingredient of logistic regression is the **sigmoid** (or *logistic*) function:

**Sigmoid Function**

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

The sigmoid function has several important properties:

- Its output is always in the open interval $(0, 1)$, regardless of the input.
- $\sigma(0) = 0.5$: at $z = 0$ the function outputs exactly one half.
- As $z \to +\infty$, $\sigma(z) \to 1$.
- As $z \to -\infty$, $\sigma(z) \to 0$.
- It is smooth and differentiable everywhere, which is essential for gradient-based optimisation.




## 2.3 Odds, Log-Odds, and Logits

The sigmoid function maps any real number $z$ to a probability in $(0, 1)$. It is natural to ask the inverse question: given a probability $\hat{p}$, what is the corresponding value of $z$? Answering this leads to the concept of **logits**, which gives logistic regression its name.

### Step 1: Odds

The **odds** of an event with probability $\hat{p}$ is the ratio of the probability that the event occurs to the probability that it does not:

**Odds**

$$\text{odds} = \frac{\hat{p}}{1 - \hat{p}} \tag{2}$$

The odds take values in $(0, +\infty)$. An odds of 1 means the event is equally likely to occur or not ($\hat{p} = 0.5$); odds greater than 1 mean the event is more likely than not; odds less than 1 mean it is less likely. Table 1 illustrates the correspondence.

Table 1: Correspondence between probability, odds, and log-odds.

| $\hat{p}$ | Odds $= \hat{p}/(1 - \hat{p})$ | Log-odds $= \log(\text{odds})$ |
|---|---|---|
| 0.01 | 0.010 | −4.60 |
| 0.10 | 0.111 | −2.20 |
| 0.25 | 0.333 | −1.10 |
| 0.50 | 1.000 | 0.00 |
| 0.75 | 3.000 | 1.10 |
| 0.90 | 9.000 | 2.20 |
| 0.99 | 99.00 | 4.60 |

### Step 2: Log-Odds (Logit)

Taking the natural logarithm of the odds maps the range $(0, +\infty)$ to the entire real line $(-\infty, +\infty)$:

**Log-Odds / Logit**

$$\text{logit}(\hat{p}) = \log \frac{\hat{p}}{1-\hat{p}} \tag{3}$$

The logit function is the **inverse of the sigmoid**: if $\hat{p} = \sigma(z)$, then $z = \text{logit}(\hat{p})$. This can be verified directly:

$$\log \frac{\sigma(z)}{1-\sigma(z)} = \log \frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}} = \log \frac{1}{e^{-z}} = \log e^{z} = z. \qquad \checkmark$$

### Step 3: The Key Insight

Combining the logit with the hypothesis function $h_\theta(x) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$ yields:

**Logistic Regression as a Linear Model for Log-Odds**

$$\log \frac{h_\theta(x)}{1-h_\theta(x)} = \boldsymbol{\theta}^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n \tag{4}$$

This is the central interpretive result of logistic regression: **the log-odds of the positive class is a linear function of the features**. In other words, logistic regression is a linear model — not for the probability $\hat{p}$ directly, but for the log-odds of $\hat{p}$. The sigmoid is simply the transformation that maps this linear score back to a probability.

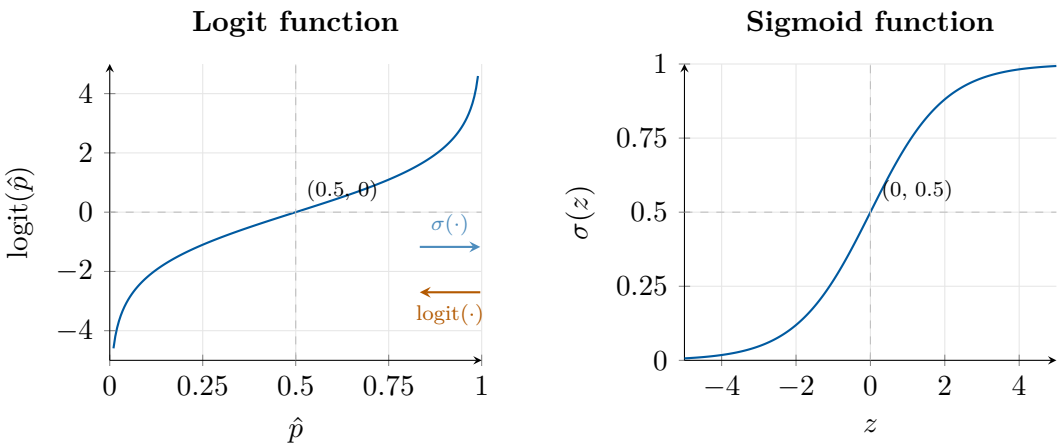



Figure 1: The logit (left) and sigmoid (right) are inverse functions of each other. The logit maps a probability $\hat{p} \in (0,1)$ to the entire real line; the sigmoid maps any real $z$ back to $(0,1)$. In logistic regression, $z = \boldsymbol{\theta}^T \mathbf{x}$ is the log-odds, and $\sigma(z)$ is the predicted probability.

**Why "logistic"?**

The name *logistic regression* comes directly from the logit transformation: the model assumes that the *log*istic (log-odds) of the outcome is a linear function of the features. The sigmoid is simply the inverse of this transformation, used to recover a

probability from the linear score.

**Entry Ticket 1**

Refer to the logit function and Equation (4).

1. A model predicts $\hat{p} = 0.75$ for some input $x$. Compute the odds and the log-odds. What is the value of $\boldsymbol{\theta}^T\mathbf{x}$ for this example?
2. If $\boldsymbol{\theta}^T\mathbf{x}$ increases by one unit, by how much do the *log-odds* change? Does this depend on the current value of $x$?
3. Explain in one sentence why logistic regression is described as a *linear* model, even though its output $\hat{p}$ is a non-linear function of $\mathbf{x}$.

## 2.4 The Hypothesis Function

For logistic regression, the hypothesis is constructed in two steps. First, compute the linear combination of the inputs:

$$z = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \boldsymbol{\theta}^T\mathbf{x}$$

Then, pass $z$ through the sigmoid function:

**Hypothesis for Logistic Regression**

$$h_\theta(x) = \sigma\left(\boldsymbol{\theta}^T\mathbf{x}\right) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T\mathbf{x}}} \tag{5}$$

**Interpretation**: $h_\theta(x)$ is the *estimated probability* that $y = 1$ given input $x$ and parameters $\theta$:

$$h_\theta(x) = \Pr(y = 1 \mid x;\, \theta)$$

Because $h_\theta(x) \in (0, 1)$, it is also true that $\Pr(y = 0 \mid x;\, \theta) = 1 - h_\theta(x)$.

## 2.5 Decision Boundary

To produce a class prediction from the probability estimate, we apply a threshold (typically 0.5):

$$\hat{y} = \begin{cases} 1 & \text{if } h_\theta(x) \geq 0.5 \\ 0 & \text{if } h_\theta(x) < 0.5 \end{cases}$$

Since $\sigma(z) \geq 0.5$ if and only if $z \geq 0$, predicting $\hat{y} = 1$ is equivalent to:

$$\boldsymbol{\theta}^T\mathbf{x} \geq 0$$

The set of all points $x$ for which $\boldsymbol{\theta}^T\mathbf{x} = 0$ is called the **decision boundary**. In two dimensions $(x_1, x_2)$, this is a straight line; in higher dimensions, it is a hyperplane. This boundary separates the region where the model predicts class 1 from the region where it predicts class 0.

**Note**

Logistic regression is a *linear* classifier: the decision boundary is always a hyperplane (a straight line in 2D). Non-linear boundaries can be obtained by adding polynomial or interaction features, just as in polynomial regression.

**Entry Ticket 2**

Refer to the hypothesis $h_\theta(x) = \sigma(\theta^T x)$ and the sigmoid function.

1. What is the output of $\sigma(z)$ when $z = 0$? What does this imply for the predicted probability when $\boldsymbol{\theta}^T\mathbf{x} = 0$?
2. If $h_\theta(x) = 0.7$, what is the model's estimated probability that $y = 0$? What class would be predicted?
3. Consider a univariate model with $\theta_0 = -3$ and $\theta_1 = 1$. For which values of $x$ does the model predict $y = 1$? Where is the decision boundary?

# 3 A Worked Example: Tumour Classification

This section illustrates the full logistic regression inference pipeline, from raw feature values to a predicted class label, using a small concrete dataset with parameters already learned.

## 3.1 Dataset and Learned Parameters

Consider a binary classification problem where the goal is to predict whether a tumour is **malignant** ($y = 1$) or **benign** ($y = 0$) based on a single feature $x_1$ (tumour size in cm). Suppose that after training, the learned parameters are:

$$\theta_0 = -8, \qquad \theta_1 = 2.$$

We wish to classify the following five new examples:

| **Example** | $x_1$ (size, cm) | **True label** $y$ |
|---|---|---|
| $\mathbf{x}^{(1)}$ | 1.0 | 0 |
| $\mathbf{x}^{(2)}$ | 3.0 | 0 |
| $\mathbf{x}^{(3)}$ | 4.0 | 1 |
| $\mathbf{x}^{(4)}$ | 5.5 | 1 |
| $\mathbf{x}^{(5)}$ | 7.0 | 1 |

## 3.2 Step 1: Locate the Decision Boundary

Before computing individual predictions, it is useful to identify the decision boundary. With $\theta_0 = -8$ and $\theta_1 = 2$, the boundary is the point $x_1^*$ where $z = \theta_0 + \theta_1 x_1 = 0$:

$$-8 + 2x_1^* = 0 \quad \Longrightarrow \quad x_1^* = 4.$$

The model predicts $\hat{y} = 1$ for $x_1 > 4$ and $\hat{y} = 0$ for $x_1 < 4$.

## 3.3 Step 2: Compute the Linear Combination $z$

For each example, compute $z^{(i)} = \theta_0 + \theta_1 x_1^{(i)} = -8 + 2x_1^{(i)}$:

| **Example** | $x_1$ | **Computation of** $z$ |
|---|---|---|
| $\mathbf{x}^{(1)}$ | 1.0 | $z = -8 + 2(1.0) = -6.0$ |
| $\mathbf{x}^{(2)}$ | 3.0 | $z = -8 + 2(3.0) = -2.0$ |
| $\mathbf{x}^{(3)}$ | 4.0 | $z = -8 + 2(4.0) = 0.0$ |
| $\mathbf{x}^{(4)}$ | 5.5 | $z = -8 + 2(5.5) = 3.0$ |
| $\mathbf{x}^{(5)}$ | 7.0 | $z = -8 + 2(7.0) = 6.0$ |

## 3.4 Step 3: Apply the Sigmoid Function

Apply $\hat{p}^{(i)} = \sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$ to obtain the estimated probability $\Pr(y = 1 \mid x^{(i)}; \theta)$:

| **Example** | $z$ | **Computation of** $\hat{p}$ | $\hat{p} = \sigma(z)$ |
|---|---|---|---|
| $\mathbf{x}^{(1)}$ | $-6.0$ | $1/(1 + e^{6.0})$ | 0.002 |
| $\mathbf{x}^{(2)}$ | $-2.0$ | $1/(1 + e^{2.0})$ | 0.119 |
| $\mathbf{x}^{(3)}$ | 0.0 | $1/(1 + e^{0})$ | 0.500 |
| $\mathbf{x}^{(4)}$ | 3.0 | $1/(1 + e^{-3.0})$ | 0.953 |
| $\mathbf{x}^{(5)}$ | 6.0 | $1/(1 + e^{-6.0})$ | 0.998 |

## 3.5 Step 4: Apply the Decision Threshold

Apply $\hat{y}^{(i)} = \mathbf{1}[\hat{p}^{(i)} \geq 0.5]$ and compare with the true label:

| **Example** | $\hat{p}$ | $\hat{y}$ | True $y$ | **Correct?** |
|---|---|---|---|---|
| $\mathbf{x}^{(1)}$ | 0.002 | 0 | 0 | **Yes** |
| $\mathbf{x}^{(2)}$ | 0.119 | 0 | 0 | **Yes** |
| $\mathbf{x}^{(3)}$ | 0.500 | 1 | 1 | **Yes** |
| $\mathbf{x}^{(4)}$ | 0.953 | 1 | 1 | **Yes** |
| $\mathbf{x}^{(5)}$ | 0.998 | 1 | 1 | **Yes** |

All five examples are correctly classified. Note that $\mathbf{x}^{(3)}$ lies exactly on the decision boundary ($z = 0$, $\hat{p} = 0.5$): this is the point of maximum uncertainty for the model.

**Entry Ticket 3**

Refer to the worked example above.

1. Suppose a new tumour has size $x_1 = 3.5$ cm. Compute $z$, $\hat{p}$, and $\hat{y}$ step by step. On which side of the decision boundary does it fall?
2. If the decision threshold is lowered from 0.5 to 0.3 (to reduce missed malig-

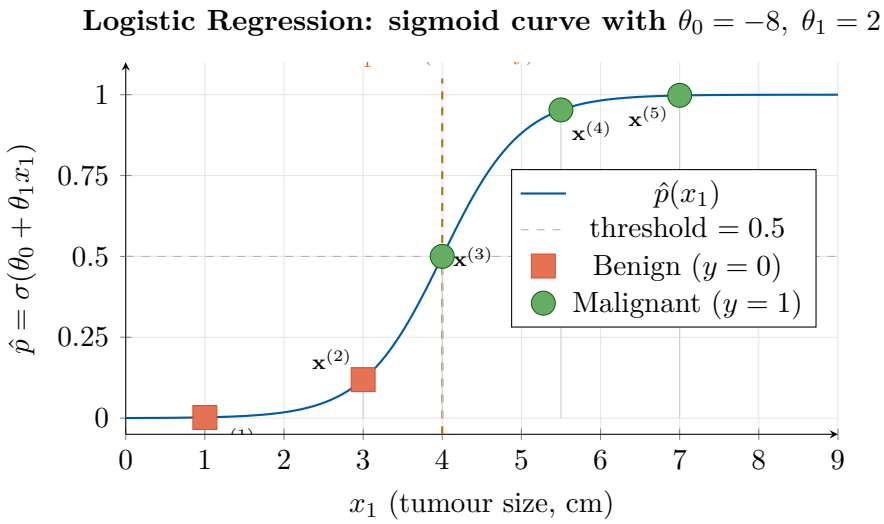



Figure 2: Sigmoid curve for the learned parameters $\theta_0 = -8$, $\theta_1 = 2$. Each training example is plotted at its estimated probability $\hat{p}$. The dashed horizontal line marks the decision threshold at 0.5; the dashed vertical line marks the decision boundary at $x_1^* = 4$. Examples to the left of the boundary are predicted benign; those to the right are predicted malignant.

nancies), how does the prediction for $\mathbf{x}^{(2)}$ ($\hat{p} = 0.119$) change, if at all? What about for a hypothetical example with $\hat{p} = 0.35$?

3. With the same parameters $\theta_0 = -8$, $\theta_1 = 2$, what tumour size $x_1$ would yield $\hat{p} = 0.9$? Hint: set $\sigma(z) = 0.9$ and solve for $z$, then for $x_1$.

# 4 The Cost Function

## 4.1 Why Not Use the MSE?

A reasonable first attempt would be to reuse the Mean Squared Error cost from linear regression:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} \left( h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

However, when $h_\theta(x) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$ is non-linear, this cost function becomes **non-convex**: it has many local minima, and gradient descent is not guaranteed to find the global minimum.

We need a different cost function, one that is convex for logistic regression and that is well-motivated from a probabilistic standpoint.

## 4.2 The Log-Loss (Binary Cross-Entropy)

The standard cost function for logistic regression is derived from the principle of **Maximum Likelihood Estimation (MLE)**: we seek the parameters $\theta$ that make the observed training data most probable.

For a single training example $(x^{(i)}, y^{(i)})$, the loss is:

**Logistic Loss (per example)**

$$\mathcal{L}\Big(h_\theta(x^{(i)}), y^{(i)}\Big) = -\,y^{(i)} \log\Big(h_\theta(x^{(i)})\Big) - \Big(1 - y^{(i)}\Big) \log\Big(1 - h_\theta(x^{(i)})\Big) \tag{6}$$

This expression is sometimes called **log-loss** or **binary cross-entropy**. Let us unpack its behaviour in each case:

- **When** $y^{(i)} = 1$: the second term vanishes, and the loss becomes $-\log(h_\theta(x^{(i)}))$. This is large when $h_\theta(x^{(i)}) \approx 0$ (wrong prediction) and approaches zero when $h_\theta(x^{(i)}) \approx 1$ (correct prediction).
- **When** $y^{(i)} = 0$: the first term vanishes, and the loss becomes $-\log(1 - h_\theta(x^{(i)}))$. This is large when $h_\theta(x^{(i)}) \approx 1$ (wrong prediction) and approaches zero when $h_\theta(x^{(i)}) \approx 0$ (correct prediction).

In both cases, the loss heavily penalises confident wrong predictions (because $\log(p) \to -\infty$ as $p \to 0$), and assigns near-zero cost to confident correct predictions.

## 4.3 The Full Cost Function

Averaging the per-example loss over all $m$ training examples:

**Logistic Regression Cost Function**

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \Big[ y^{(i)} \log\Big(h_\theta(x^{(i)})\Big) + \Big(1 - y^{(i)}\Big) \log\Big(1 - h_\theta(x^{(i)})\Big) \Big] \tag{7}$$

**Convexity**

Unlike the MSE applied to a sigmoid output, the log-loss cost function in Equation (7) is **convex** in $\theta$. This means it has a unique global minimum, and gradient descent is guaranteed to converge to it (for a suitable learning rate).

## 4.4 Optimisation Objective

Our goal is:

**Optimisation Objective**

$$\underset{\theta}{\text{minimise}} \quad J(\theta) \tag{8}$$

**Entry Ticket 4**

Refer to the logistic regression cost function $J(\theta)$.

1. Suppose a model predicts $h_\theta(x^{(i)}) = 0.01$ for an example where $y^{(i)} = 1$. Is the loss high or low? Why is such a prediction heavily penalised?
2. Consider the two cases $y^{(i)} = 1$ and $y^{(i)} = 0$ separately. Show that Equa-

tion (6) reduces to a single logarithm in each case and describe the behaviour of each.

3. Why can we not use the Mean Squared Error cost function (from linear regression) for logistic regression? What issue arises?

# 5 Parameter Learning: Gradient Descent

## 5.1 Computing the Gradient

To apply gradient descent, we need the partial derivative of $J(\theta)$ with respect to each parameter $\theta_j$. The derivation uses the chain rule and the useful identity:

$$\frac{d}{dz}\sigma(z) = \sigma(z)\,(1 - \sigma(z))$$

After carrying out the differentiation (see Appendix A for the full derivation), the result is remarkably clean:

**Partial Derivative of the Logistic Cost**

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)} \tag{9}$$

where $x_0^{(i)} = 1$ by convention (for the intercept term $\theta_0$).

**Structural similarity to linear regression**

Compare Equation (9) with the gradient of the MSE cost from linear regression:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

The expressions are *identical in form*! The key difference is that in logistic regression, $h_\theta(x^{(i)}) = \sigma(\boldsymbol{\theta}^T\mathbf{x}^{(i)})$ is a non-linear function of $\theta$, whereas in linear regression it is linear. The update equations *look* the same, but they compute different quantities.

## 5.2 The Gradient Descent Algorithm

**Gradient Descent for Logistic Regression**

Repeat until convergence:

$$\theta_j \leftarrow \theta_j - \alpha \cdot \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)} \qquad \text{for all } j = 0, 1, \ldots, n \tag{10}$$

with all $\theta_j$ updated **simultaneously**.

As in linear regression:

- $\alpha > 0$ is the **learning rate**.
- The residual $h_\theta(x^{(i)}) - y^{(i)}$ drives the update: if the model over-predicts, the parameters are adjusted downward, and vice versa.
- Updates must be applied simultaneously using temporary variables.

**Entry Ticket 5**

Refer to the gradient descent update rule for logistic regression (Equation (10)).

1. The gradient formula looks identical to the one from linear regression. What is fundamentally different about how $h_\theta(x^{(i)})$ is computed in each case?
2. Suppose that for every training example, the model predicts exactly the correct label. What would the gradient $\partial J/\partial \theta_j$ equal? What would happen to the parameters in the next update?
3. What is the role of the factor $x_j^{(i)}$ in the update for $\theta_j$? Why does the update for $\theta_0$ use $x_0^{(i)} = 1$?

## 5.3 Worked Example: Single Update Step

Consider a binary classification problem with a single feature $x$ and training set from Table 2.

Table 2: Small binary classification training set.

| $i$ | $x^{(i)}$ | $y^{(i)}$ |
|---|---|---|
| 1 | 1.0 | 0 |
| 2 | 2.0 | 0 |
| 3 | 3.5 | 1 |
| 4 | 5.0 | 1 |

Suppose we start with $\theta_0 = 0$, $\theta_1 = 0$ and use $\alpha = 0.1$.

### Step 1: Compute predictions

With $\theta_0 = \theta_1 = 0$, the linear combination is $z^{(i)} = 0 + 0 \cdot x^{(i)} = 0$ for all examples, so:

$$h_\theta(x^{(i)}) = \sigma(0) = 0.5 \qquad \text{for all } i$$

### Step 2: Compute residuals

| $i$ | $h_\theta(x^{(i)})$ | $y^{(i)}$ | residual $= h_\theta - y^{(i)}$ |
|---|---|---|---|
| 1 | 0.5 | 0 | $+0.5$ |
| 2 | 0.5 | 0 | $+0.5$ |
| 3 | 0.5 | 1 | $-0.5$ |
| 4 | 0.5 | 1 | $-0.5$ |

**Step 3: Compute gradients ($m = 4$)**

$$\begin{aligned}
\frac{\partial J}{\partial \theta_0} &= \frac{1}{4}\Big[(0.5)(1) + (0.5)(1) + (-0.5)(1) + (-0.5)(1)\Big] = \frac{1}{4}(0) = 0 \\
\frac{\partial J}{\partial \theta_1} &= \frac{1}{4}\Big[(0.5)(1.0) + (0.5)(2.0) + (-0.5)(3.5) + (-0.5)(5.0)\Big] \\
&= \frac{1}{4}\Big[0.5 + 1.0 - 1.75 - 2.5\Big] = \frac{1}{4}(-2.75) = -0.6875
\end{aligned}$$

**Step 4: Update parameters**

$$\begin{aligned}
\theta_0 &\leftarrow 0 - 0.1 \times 0 = 0 \\
\theta_1 &\leftarrow 0 - 0.1 \times (-0.6875) = 0.06875
\end{aligned}$$

After one step, $\theta_1$ has increased from 0 to 0.06875, making the slope of the decision boundary more positive — exactly the direction needed to assign higher probabilities to the positive-class examples ($x = 3.5$ and $x = 5.0$).

# 6 The Decision Boundary in Detail

## 6.1 Linear Decision Boundary

In the univariate case, the decision boundary is the value of $x$ for which $h_\theta(x) = 0.5$, i.e., $\theta_0 + \theta_1 x = 0$:

$$x^* = -\frac{\theta_0}{\theta_1}$$

For example, if $\theta_0 = -3$ and $\theta_1 = 1$, then $x^* = 3$: the model predicts class 1 for all $x > 3$ and class 0 for all $x < 3$.

In the bivariate case (features $x_1$ and $x_2$), the boundary is the line:

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0 \quad \Longleftrightarrow \quad x_2 = -\frac{\theta_0}{\theta_2} - \frac{\theta_1}{\theta_2} x_1$$

## 6.2 Non-Linear Decision Boundaries

Logistic regression can model non-linear decision boundaries by introducing **polynomial features**. For example, including the features $x_1^2$ and $x_2^2$:

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 = 0$$

This defines an ellipse (or other conic section) as the decision boundary, allowing the classifier to separate data that is not linearly separable in the original feature space.

**Entry Ticket 6**

Refer to the decision boundary of logistic regression.

1. Given $\theta_0 = -6$, $\theta_1 = 1$, and $\theta_2 = 1$ (bivariate model), write the equation of the decision boundary. What is the geometric shape of the region predicted as class 1?
2. Suppose we add polynomial features $x_1^2$ and $x_2^2$, and learn $\theta_0 = -1$, $\theta_3 = 1$, $\theta_4 = 1$ (all other $\theta_j = 0$). What geometric shape does the decision boundary form?
3. What is the prediction for an example that lies *exactly on* the decision boundary? Is the model certain about this prediction?

# 7 Regularisation

## 7.1 The Problem of Overfitting

When a model has too many parameters relative to the number of training examples, it can **overfit**: it fits the training data very closely but generalises poorly to new, unseen data. In logistic regression, this typically manifests as a highly complex, jagged decision boundary that perfectly separates the training points but fails on test points.

The opposite extreme is **underfitting** (high bias): the model is too simple to capture the structure in the data. The goal is to find the right balance.

## 7.2 Regularisation

**Regularisation** addresses overfitting by adding a penalty term to the cost function that discourages large parameter values. Intuitively, large weights lead to sharp, sensitive decision boundaries that are prone to overfitting; keeping weights small promotes smoother, more general models.

The most common form is **L2 regularisation** (also called *Ridge*):

**Regularised Logistic Regression Cost**

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(h_\theta(x^{(i)})\right) + \left(1 - y^{(i)}\right)\log\left(1 - h_\theta(x^{(i)})\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2 \quad (11)$$

Two important conventions:

- The regularisation term penalises $\theta_1, \ldots, \theta_n$ but **not** $\theta_0$ (the bias/intercept). Penalising the bias would shift all predictions indiscriminately without reducing complexity.
- $\lambda \geq 0$ is the **regularisation parameter** (a hyperparameter). Larger $\lambda$ increases the penalty, pushing weights toward zero and producing a simpler model. Setting $\lambda = 0$ recovers the unregularised cost.

### 7.3 Regularised Gradient Descent

Adding the regularisation term changes the gradient for $j \geq 1$:

**Regularised Gradient Descent Updates**

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^{m} \left( h_\theta(x^{(i)}) - y^{(i)} \right) x_0^{(i)} \tag{12}$$

$$\theta_j \leftarrow \theta_j \left( 1 - \frac{\alpha\lambda}{m} \right) - \frac{\alpha}{m} \sum_{i=1}^{m} \left( h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \qquad (j \geq 1) \tag{13}$$

The factor $\left(1 - \frac{\alpha\lambda}{m}\right)$ in Equation (13) is slightly less than 1 (for small $\alpha\lambda/m$), so at each step $\theta_j$ is first *shrunk* toward zero before the gradient correction is applied. This is the regularisation effect in action.

**Entry Ticket 7**

Refer to regularised logistic regression.

1. Why is $\theta_0$ excluded from the regularisation penalty? What would happen if we included it?
2. If $\lambda$ is set very large (e.g. $\lambda = 10{,}000$), what values will the parameters $\theta_1, \ldots, \theta_n$ be pushed toward? What would the resulting hypothesis look like geometrically?
3. The factor $(1 - \alpha\lambda/m)$ appears in the update for $\theta_j$. Under what conditions (on $\alpha$, $\lambda$, $m$) might this factor become negative, and why would that be problematic?

## 8 Prediction and Evaluation

### 8.1 Making Predictions

Given learned parameters $\theta$, prediction for a new input $x$ proceeds in two steps:

1. Compute the estimated probability: $\hat{p} = h_\theta(x) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$.
2. Apply the decision threshold: $\hat{y} = \mathbf{1}[\hat{p} \geq 0.5]$.

The threshold of 0.5 is the default, but it can be adjusted depending on the application. For example, in a medical screening context one might lower the threshold (e.g. 0.3) to reduce the rate of missed positives (false negatives), at the cost of more false positives.

### 8.2 Evaluation Metrics

Because the classes may be **imbalanced** (far fewer positive examples than negative, or vice versa), raw accuracy (fraction of correctly classified examples) can be misleading. The **confusion matrix** organises predictions into four categories:

| | **Predicted** $\hat{y} = 1$ | **Predicted** $\hat{y} = 0$ |
|---|---|---|
| **Actual** $y = 1$ | True Positive (TP) | False Negative (FN) |
| **Actual** $y = 0$ | False Positive (FP) | True Negative (TN) |

From these, the most commonly reported metrics are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(of all predicted positives, how many are truly positive?)}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad \text{(of all actual positives, how many were detected?)}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{(harmonic mean of precision and recall)}$$

**Precision vs. Recall Trade-off**

Lowering the decision threshold increases recall (fewer actual positives are missed) but decreases precision (more false alarms). Raising the threshold has the opposite effect. The best threshold depends on the relative cost of false positives versus false negatives in the application domain.

**Entry Ticket 8**

Refer to the evaluation of a logistic regression classifier.

1. A classifier is evaluated on a dataset with 95 negative examples and 5 positive examples. A trivial model that always predicts $\hat{y} = 0$ achieves 95% accuracy. Compute its precision and recall. Why is accuracy alone misleading here?
2. In a cancer screening application, which is more costly: a false negative (missed cancer) or a false positive (unnecessary further tests)? Should the decision threshold be raised or lowered?
3. What does an F1-Score of 1.0 indicate? And what does 0.0 indicate?

# 9 Summary

The table below summarises the key components of logistic regression.

| **Component** | **Expression** |
|---|---|
| Sigmoid function | $\sigma(z) = \dfrac{1}{1+e^{-z}}$ |
| Hypothesis | $h_\theta(x) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$ |
| Cost function | $J(\theta) = -\dfrac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log h_\theta(x^{(i)}) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))\right]$ |
| Regularised cost | $J(\theta) + \dfrac{\lambda}{2m}\sum_{j=1}^{n}\theta_j^2$ |
| Gradient | $\dfrac{\partial J}{\partial \theta_j} = \dfrac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$ |
| GD update ($j=0$) | $\theta_0 \leftarrow \theta_0 - \dfrac{\alpha}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)$ |
| GD update ($j \geq 1$) | $\theta_j \leftarrow \theta_j\left(1 - \dfrac{\alpha\lambda}{m}\right) - \dfrac{\alpha}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$ |
| Prediction | $\hat{y} = 1$ if $h_\theta(x) \geq 0.5$, else $\hat{y} = 0$ |

**Key Takeaways**

1. Logistic regression is used for **binary classification**: the target is $y \in \{0, 1\}$.
2. The hypothesis $h_\theta(x) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$ outputs a probability in $(0, 1)$ by applying the sigmoid function to the linear combination of features.
3. The cost function is the **log-loss** (binary cross-entropy), which is convex in $\theta$ and derived from maximum likelihood estimation.
4. The gradient of the log-loss has the **same algebraic form** as that of the MSE in linear regression, making the gradient descent update rules structurally identical.
5. **L2 regularisation** (with parameter $\lambda$) prevents overfitting by penalising large parameter values, shrinking them toward zero at each update step.
6. Evaluation goes beyond accuracy: **precision, recall, and F1-score** are essential when classes are imbalanced.

# A Appendix: Deriving the Gradient of the Log-Loss

This appendix provides the full derivation of the gradient in Equation (9).

We wish to compute $\dfrac{\partial J}{\partial \theta_j}$ for a single training example $(x, y)$, then generalise by summing over all examples.

The per-example loss is:

$$\mathcal{L} = -y\log(h_\theta(x)) - (1-y)\log(1-h_\theta(x))$$

Denote $\hat{p} = h_\theta(x) = \sigma(z)$, where $z = \boldsymbol{\theta}^T \mathbf{x}$.

**Step 1: Derivative of $\mathcal{L}$ with respect to $\hat{p}$**

$$\frac{\partial \mathcal{L}}{\partial \hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}} = \frac{\hat{p}-y}{\hat{p}(1-\hat{p})}$$

**Step 2: Derivative of $\hat{p}$ with respect to $z$**

Using the sigmoid derivative identity:

$$\frac{d\hat{p}}{dz} = \sigma(z)\,(1-\sigma(z)) = \hat{p}(1-\hat{p})$$

**Step 3: Derivative of $z$ with respect to $\theta_j$**

$$\frac{\partial z}{\partial \theta_j} = x_j$$

**Step 4: Combine via the chain rule**

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \frac{\partial \mathcal{L}}{\partial \hat{p}} \cdot \frac{d\hat{p}}{dz} \cdot \frac{\partial z}{\partial \theta_j} = \frac{\hat{p}-y}{\hat{p}(1-\hat{p})} \cdot \hat{p}(1-\hat{p}) \cdot x_j = (\hat{p}-y)\,x_j = (h_\theta(x)-y)\,x_j$$

The $\hat{p}(1-\hat{p})$ terms cancel perfectly, yielding a clean linear expression in the residual. Summing over all $m$ examples and dividing by $m$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)x_j^{(i)} \qquad \square$$

This confirms Equation (9) and explains why the gradient update rules for logistic and linear regression share the same algebraic form despite using different hypothesis functions.